

Gestione dei Permessi in Linux

1. Introduzione

Questo report descrive le attività svolte durante l'esercitazione pratica sulla gestione dei permessi in un sistema Linux. L'obiettivo era configurare i permessi di lettura, scrittura ed esecuzione su file e directory, verificando concretamente il comportamento del sistema quando un utente non autorizzato tenta di accedere a risorse protette.

2. Scenario Scelto

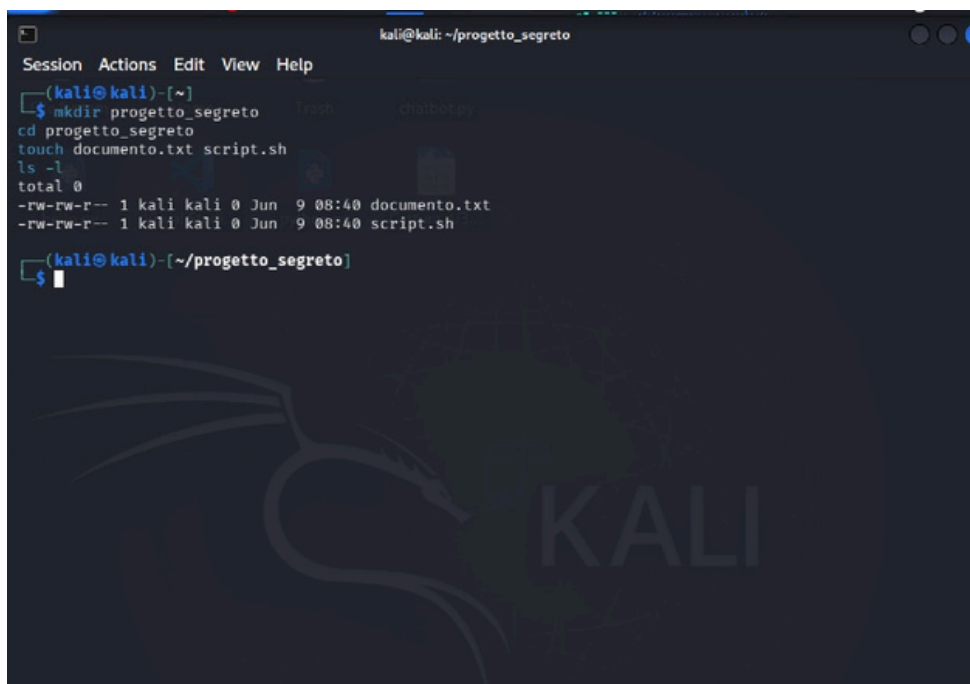
Ho scelto di simulare uno scenario ispirato al laboratorio Windows Server del giorno precedente: una directory chiamata `progetto_segreto` contenente due file con livelli di accesso differenti.

Il primo file, `documento.txt`, rappresenta un documento riservato che solo il proprietario può modificare, mentre i membri del gruppo possono leggerlo. Il secondo file, `script.sh`, rappresenta uno script eseguibile che il proprietario e il gruppo possono usare, ma dal quale gli utenti esterni sono completamente esclusi.

Questa scelta permette di dimostrare tutte e tre le tipologie di permesso — lettura, scrittura ed esecuzione — su due file con configurazioni diverse, rendendo l'esercizio più completo e realistico.

3. Creazione della Directory e dei File

La prima operazione è stata creare la directory `progetto_segreto` nella home dell'utente `kali`, usando il comando `mkdir`. Successivamente, tramite il comando `cd`, mi sono spostato al suo interno e ho creato i due file con il comando `touch`. Il comando `ls -l` ha mostrato i permessi assegnati automaticamente dal sistema al momento della creazione.

A screenshot of a terminal window in Kali Linux. The window title is 'kali@kali: ~/progetto_segreto'. The terminal shows the following commands and output:

```
Session Actions Edit View Help
(kali@kali)-[~]
$ mkdir progetto_segreto
cd progetto_segreto
touch documento.txt script.sh
ls -l
total 0
-rw-rw-r-- 1 kali kali 0 Jun  9 08:40 documento.txt
-rw-rw-r-- 1 kali kali 0 Jun  9 08:40 script.sh
(kali@kali)-[~/progetto_segreto]
$
```

The background of the terminal features a faint Kali Linux dragon logo and the word 'KALI'.

I permessi iniziali `-rw-rw-r--` indicano che sia il proprietario che il gruppo possono leggere e scrivere, mentre gli altri utenti hanno solo il permesso di lettura. Non è presente alcun permesso di esecuzione. Questa configurazione di default non è adatta allo scenario che voglio simulare, quindi è necessario modificarla.

4. Modifica dei Permessi con `chmod`

Ho utilizzato il comando `chmod` con la notazione numerica ottale per impostare i permessi desiderati su ciascun file. La notazione numerica è composta da tre cifre: la prima rappresenta i permessi del proprietario, la seconda quelli del gruppo, la terza quelli degli altri utenti. Ogni cifra è la somma dei valori dei permessi: lettura vale 4, scrittura vale 2, esecuzione vale 1.

4.1 Permessi su `documento.txt` (`chmod 640`)

Il valore 640 assegna al proprietario lettura e scrittura ($6 = 4+2$), al gruppo la sola lettura (4), e agli altri nessun permesso (0). Questa configurazione ha senso per un documento riservato: chi lo ha creato può modificarlo, i colleghi del gruppo possono consultarlo, ma nessun estraneo può accedervi.

4.2 Permessi su `script.sh` (`chmod 750`)

Il valore 750 assegna al proprietario lettura, scrittura ed esecuzione completa ($7 = 4+2+1$), al gruppo lettura ed esecuzione ($5 = 4+1$) senza possibilità di modifica, e agli altri nessun permesso (0). Uno script di automazione deve poter essere eseguito dal gruppo, ma nessuno al di fuori deve poterlo vedere o avviare.

```
(kali@kali)-[~/progetto_segreto]
$ chmod 750 script.sh
chmod 640 documento.txt
ls -l
total 0
-rw-r----- 1 kali kali 0 Jun  9 08:40 documento.txt
-rwxr-x--- 1 kali kali 0 Jun  9 08:40 script.sh

(kali@kali)-[~/progetto_segreto]
$
```

5. Verifica e Test dei Permessi

Dopo aver configurato i permessi, ho eseguito una serie di test per verificarne il corretto funzionamento. I test sono stati effettuati in due fasi: prima come utente proprietario kali, poi come utente testuser creato appositamente per simulare un accesso non autorizzato.

5.1 Test come proprietario (utente kali)

Come proprietario, ho scritto del testo nel documento usando il comando echo, verificato la lettura con cat, e avviato lo script con ./script.sh. Tutte le operazioni sono andate a buon fine, confermando che i permessi del proprietario funzionano correttamente.

```
(kali@kali)-[~/progetto_segreto]
$ echo "dati riservati" > documento.txt
cat documento.txt
./script.sh
dati riservati

(kali@kali)-[~/progetto_segreto]
$
```

5.2 Test come utente non autorizzato (testuser)

Ho creato un nuovo utente testuser con il comando useradd e ho tentato di accedere ai file simulando la sua identità tramite sudo -u. Il sistema ha bloccato tutti i tentativi: lettura del documento, esecuzione dello script e scrittura nel documento hanno restituito il messaggio Permission denied in tutti e tre i casi.

```
kali@kali: ~/progetto_segreto

Session Actions Edit View Help

(kali@kali)-[~]
$ mkdir progetto_segreto
$ cd progetto_segreto
$ touch documento.txt script.sh
$ ls -l
total 0
-rw-rw-r-- 1 kali kali 0 Jun 9 08:40 documento.txt
-rw-rw-r-- 1 kali kali 0 Jun 9 08:40 script.sh

(kali@kali)-[~/progetto_segreto]
$ chmod 750 script.sh
$ chmod 640 documento.txt
$ ls -l
total 0
-rw-r-- 1 kali kali 0 Jun 9 08:40 documento.txt
-rwxr-x 1 kali kali 0 Jun 9 08:40 script.sh

(kali@kali)-[~/progetto_segreto]
$ echo "dati riservati" > documento.txt
$ cat documento.txt
$ ./script.sh
dati riservati

(kali@kali)-[~/progetto_segreto]
$ sudo useradd testuser
$ sudo -u testuser cat /home/kali/progetto_segreto/documento.txt
[sudo] password for kali:
cat: /home/kali/progetto_segreto/documento.txt: Permission denied
$ sudo -u testuser ./script.sh
[sudo] password for kali:
cat: /home/kali/progetto_segreto/documento.txt: Permission denied
$ sudo -u testuser bash -c 'echo "tentativo" >> /home/kali/progetto_segreto/documento.txt'
bash: line 1: /home/kali/progetto_segreto/documento.txt: Permission denied

(kali@kali)-[~/progetto_segreto]
$
```

6. Analisi dei Risultati

I risultati ottenuti confermano che il sistema di permessi di Linux funziona in modo preciso e granulare. Ogni file ha risposto esattamente nel modo previsto in base alla configurazione impostata con `chmod`.

Per `documento.txt` con permessi 640, il proprietario ha potuto leggere e scrivere senza problemi. `testuser`, non appartenendo al gruppo `kali` e rientrando quindi nella categoria 'altri', ha ricevuto accesso negato sia in lettura che in scrittura, come atteso dal valore 0 nell'ultima cifra.

Per `script.sh` con permessi 750, il proprietario ha potuto eseguire lo script regolarmente. Anche in questo caso `testuser` non ha potuto né leggere né eseguire il file, ricevendo `Permission denied` in entrambi i tentativi.

Vale la pena notare il parallelo con quanto visto nella sessione precedente su Windows Server: lì i permessi venivano assegnati tramite interfaccia grafica a gruppi come `Mitiche` e `Robot`, qui vengono gestiti da riga di comando con `chmod`. Il concetto sottostante è identico — il principio del privilegio minimo, che prevede di dare a ciascun utente solo i permessi strettamente necessari al proprio ruolo.

7. Conclusioni

L'esercitazione ha permesso di mettere in pratica in modo concreto la gestione dei permessi in Linux, uno degli aspetti fondamentali della sicurezza di un sistema operativo. Attraverso pochi comandi — `mkdir`, `touch`, `chmod` e `ls -l` — è stato possibile costruire uno scenario realistico e verificarne il funzionamento con test pratici.

La scelta di usare due file con configurazioni diverse ha permesso di esplorare tutte e tre le tipologie di permesso e di capire come il sistema distingua tra proprietario, gruppo e altri utenti. I test con `testuser` hanno dimostrato che i blocchi configurati funzionano esattamente come previsto, rendendo i file inaccessibili a chiunque non sia autorizzato.